

Algorytmy ewolucyjne i genetyczne

WSI – ćwiczenie 1

Autor: Kacper Skrodzki

Adnotacja do ograniczeń twardych

W obecnej wersji kodu jest zaimplementowane ograniczenie twarde, które ma na celu zapewnienie, że żaden osobnik nie będzie mógł używać silników przez ponad 300 sekund (tzn. osobnik może mieć maksymalnie 300 jedynek).

Ograniczenie te zostało zaimplementowane w następujących obszarach:

- **Generowanie pierwszej populacji** – funkcja losująca zapewnia nie przekroczenie limitu 300 jedynek dla osobnika
- **Krzyżowanie** – jeśli wybrany punkt krzyżowania doprowadzi do powstania choć jednego potomka z przekroczonym limitem, to jest ciągle wybierany nowy punkt aż krzyżowanie zajdzie bez problemu
- **Mutacja** – w momencie mutacji zera w jedynekę jest sprawdzane, czy nie zostanie przekroczony limit. Jeśli zostanie naruszony to zaniechuje się mutacji tego bitu, w przeciwnym razie dochodzi do mutacji i aktualizuje się obecna liczba jedynek

Jakie jest trywialne rozwiązanie? Jakie jest ograniczenie dolne?

Przedstawiony nam problem optymalizacyjny opiera się na możliwej sytuacji z prawdziwego życia. Zanim przejdziemy do omawiania eksperymentów z napisanym przeze mnie programem z algorytmem genetycznym, warto pomyśleć o trywialnym rozwiązaniu przedstawionego problemu optymalizacyjnego.

Pierwsza myśl jaka przychodzi do głowy to taki plan działania maszyny latającej, gdzie na początku sterowania wznosi się cały czas do przodu i do góry, a potem wyłącza się silniki i pozwala się maszynie na lądowanie. Jest to proste rozwiązanie, które nie gwarantuje celu 350 metrów od startu, ale ukazuje ścieżkę jaką mogą podążać znalezione rozwiązania.

Ponadto warto zastanowić się jakie jest górne ograniczenie wyniku. Po krótkim namyśle można stwierdzić, że będzie się ono znajdowało gdzieś blisko zera. Nie w samym zerze, gdyż jest prawie niemożliwe wylądowanie idealnie 350 metrów od startu co do centymetra.

Cel eksperymentów

Celem eksperymentów jest zbadanie wpływu poszczególnych hiperparametrów na efektywność wyszukiwania optimum globalnego dla zadanego problemu przy użyciu algorytmu genetycznego. Badane parametry to:

- Prawdopodobieństwo mutacji
- Prawdopodobieństwo krzyżowania
- Liczba osobników w populacji
- Ilość danego FES-u

Ponadto będzie badany przebieg znajdowania optimum, czyli kiedy i jakie optimum znaleziono w trakcie działania programu.

Aby móc przeprowadzić wymienione wyżej eksperymenty najpierw trzeba było znaleźć odpowiedni zestaw hiperparametrów, dla którego średnia wartość wyników zwracanych przez program mieściła się ponad wartość -4.

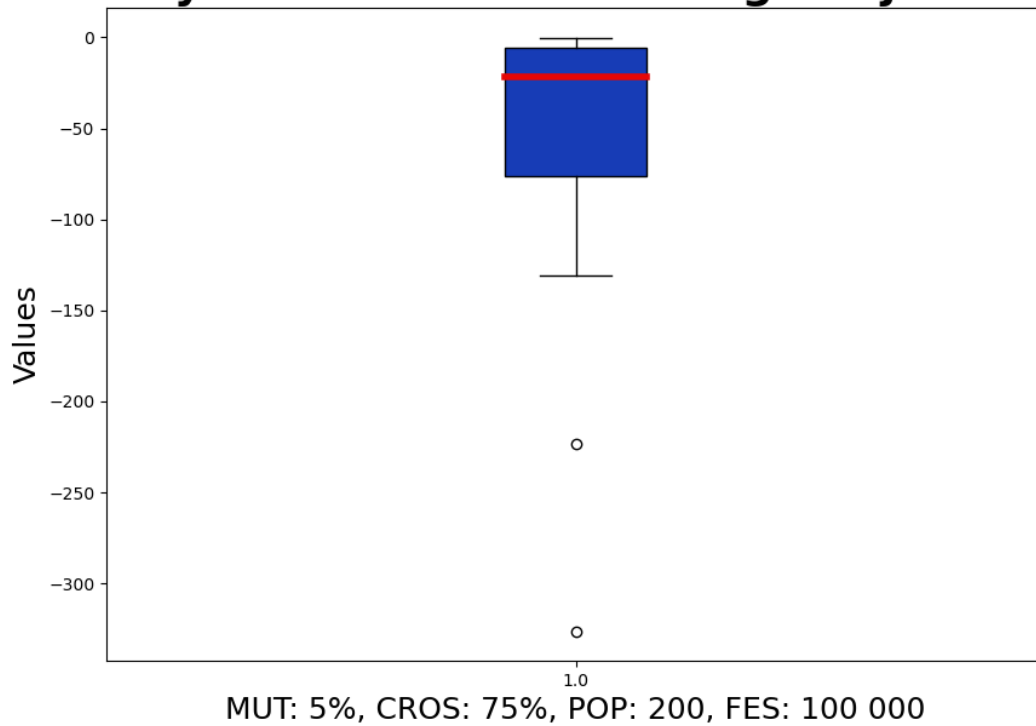
Znajdowanie bazy pod eksperymenty

Aby móc w ogóle zacząć wymienione powyżej eksperymenty należy najpierw znaleźć taki zestaw parametrów, dla których średni wynik osiągał wartości powyżej -4. W tym celu przetestowałem różne zestawy parametrów dwudziestokrotnie i wyrysowałem do nich wykresy.

Objaśnienie skrótów:

- ❖ **MUT** → Prawdopodobieństwo mutacji
- ❖ **PROB** → Prawdopodobieństwo krzyżowania
- ❖ **POP** → Rozmiar początkowej populacji:
- ❖ **FES** → Liczba możliwych ewaluacji

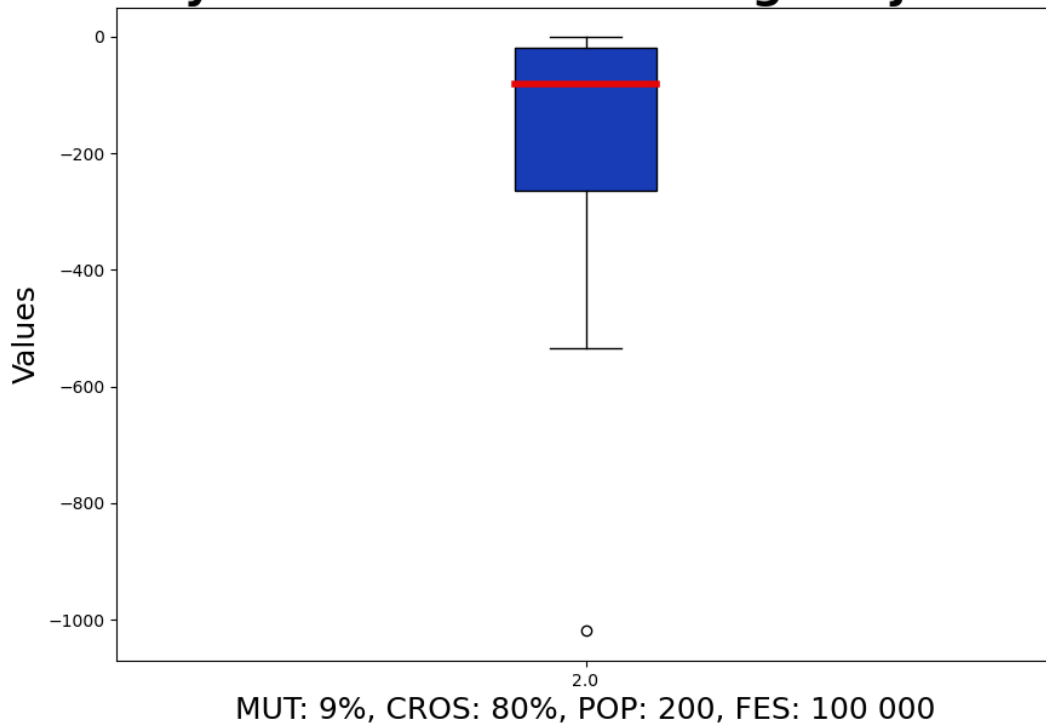
Wyniki dokładności konfiguracji nr 1



Wnioski:

Konfiguracja nr 1 dała podstawę do eksperymentowania z parametrami, aby uzyskać lepszą dokładność. Prawdopodobną przyczyną dużego rozrzutu wyników jest mała ilość pokoleń, przez które przeszła ewolucja – $100\,000 / 200 = 500$ pokoleń albo zbyt mała różnorodność osobników.

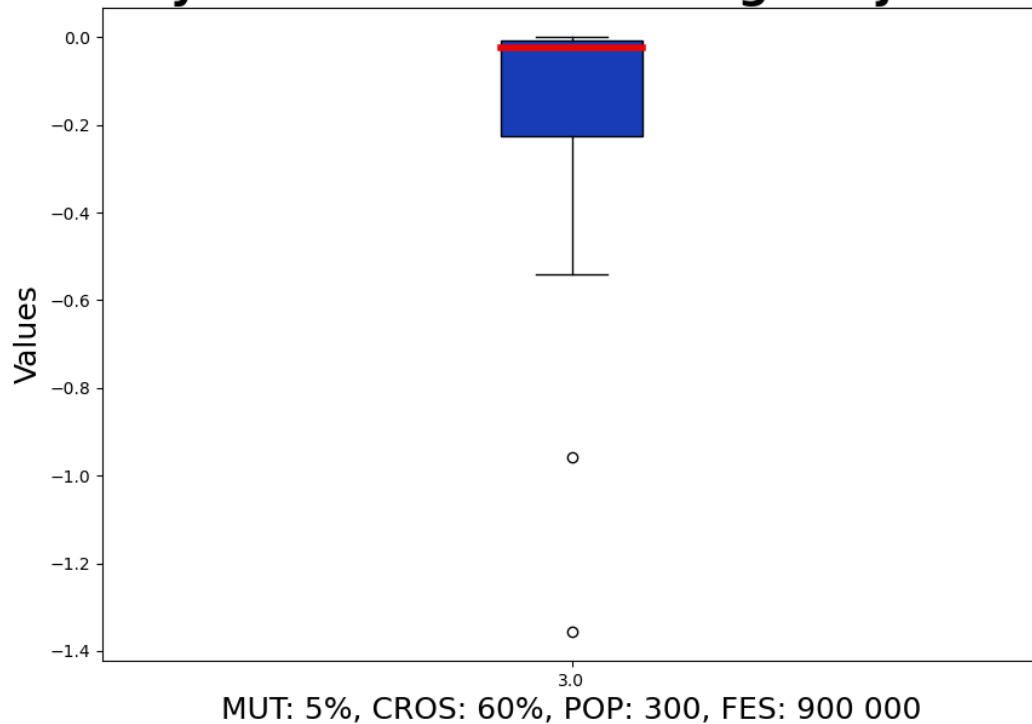
Wyniki dokładności konfiguracji nr 2



Wnioski:

Konfiguracja nr 2 odpowiada na pytanie czy zwiększenie wartości MUT i CROS zmniejszy rozrzut wyników przy zachowaniu tej samej populacji co konfiguracja nr 1. Wychodzi jednak na to, że rozrzut wartości jeszcze bardziej się zwiększył.

Wyniki dokładności konfiguracji nr 3



Wnioski:

Główną zmianą konfiguracji nr 3 jest to, że zwiększyła się liczba pokoleń poddanych ewolucji ($900\,000 / 300 = 1000$ pokoleń) oraz liczba osobników na pokolenie. Dodatkowo lekko zmniejszono parametr CROS do 60%. Ta kombinacja dała świetne wyniki, lepsze niż początkowe założenia.

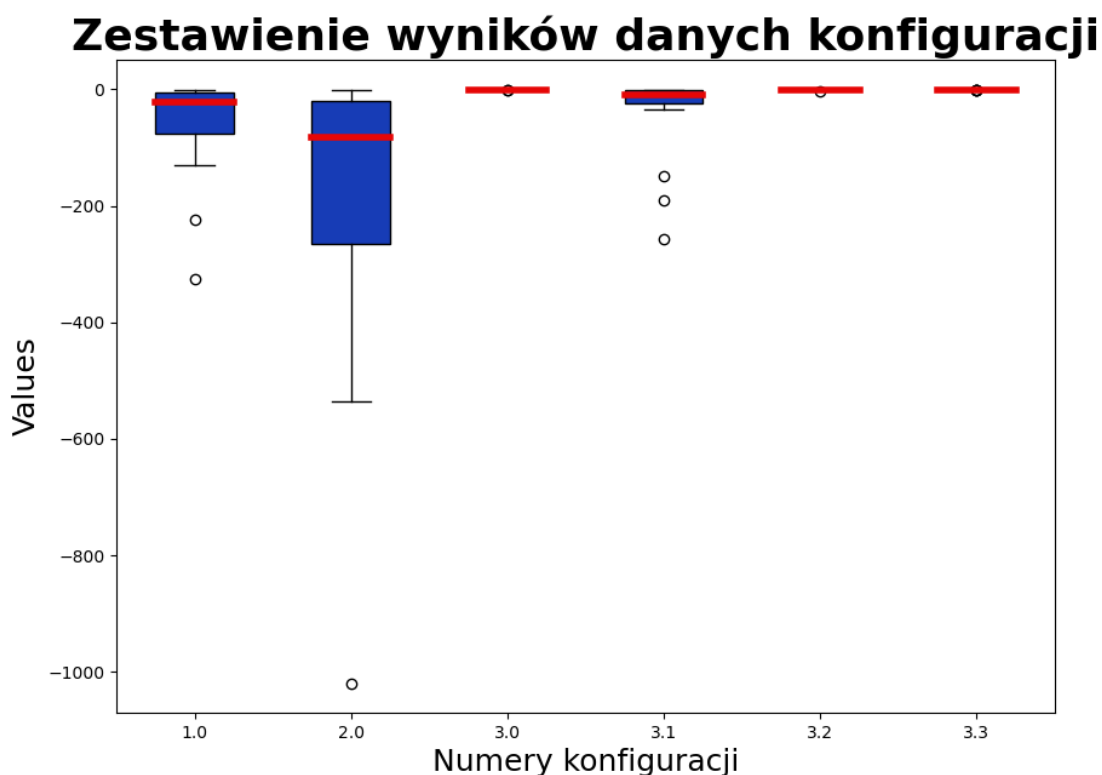
Jednakże tak dobra dokładność znajdowania optimum jest okupiona dość dużym czasem przeszukiwania dostępnej przestrzeni rozwiązań. Dlatego dodatkowo przetestowano trzy warianty konfiguracji nr 3:

Konfiguracja 3.1: MUT = 5%, CROS = 60%, POP = 300, FES = 300 000

Konfiguracja 3.2: MUT = 5%, CROS = 60%, POP = 300, FES = 450 000

Konfiguracja 3.3: MUT = 5%, CROS = 60%, POP = 300, FES = 600 000

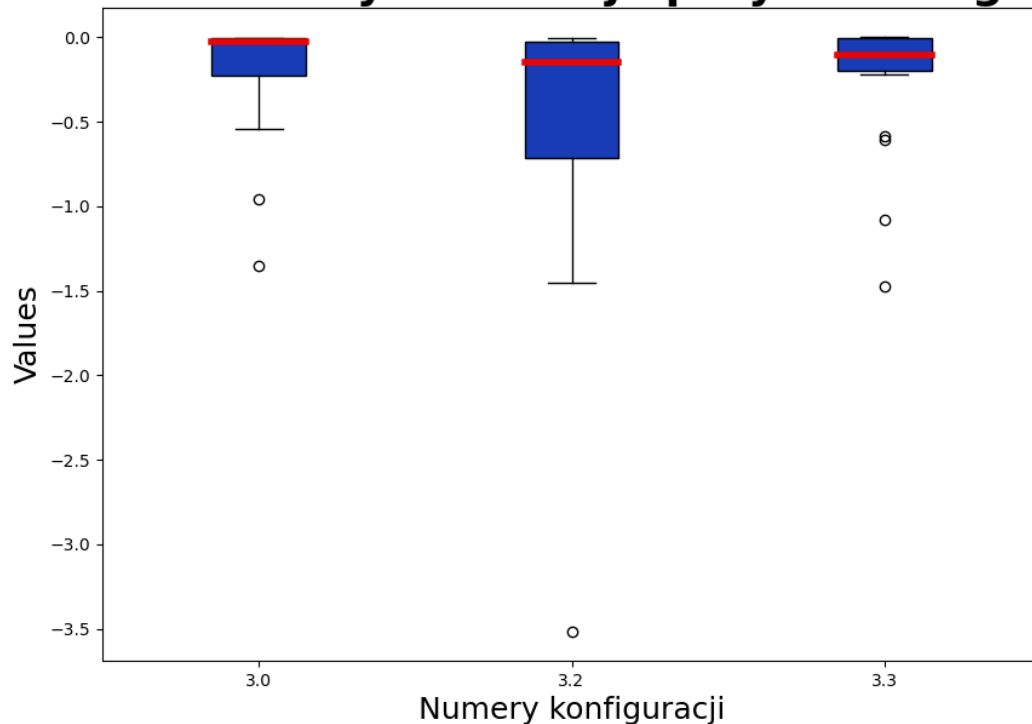
Na poniższym wykresie przedstawiono wszystkie wyniki jak dotąd wykonanych testów konfiguracji.



Wnioski:

Na pierwszy rzut oka można zauważyć, że konfiguracje numer 3, 3.2 i 3.3 wiodą prymat w porównaniu do innych przetestowanych konfiguracji. Jednak aby móc określić, która z tych konfiguracji jest najlepsza trzeba wydzielić je na osobnym wykresie o mniejszej skali.

Zestawienie wyników najlepszych konfiguracji



Wnioski:

Porównując z bliska kandydatów na najlepszą konfigurację można zauważyć zbliżoną efektywność, gdzie tylko konfiguracja 3.2 jest najbardziej rozstrzelona, ale nadal mieści się w zadanym zakresie powyżej -4.

Po analizie wykresów wybrano konfigurację nr 3.3 ze względu na mniejsze wahania niż konfiguracja 3.2 przy jednoczesnym zmniejszeniu czasu przeszukiwania, które było użyte w konfiguracji nr 3.

Podsumowanie

Po testach paru konfiguracji udało się natrafić na taki zestaw parametrów, który pozwala na określenie optimum rzędu dokładności większej niż -1. Wadą tego zestawu jest jego czas trwania przeszukiwania danej przestrzeni, lecz dokładność wyników jest warta tego.

Wybrana konfiguracja: MUT = 5%, CROS = 60%, POP = 300, FES = 600 000

Przebieg i wyniki eksperymentów

W tym przedziale zajmę się omawianiem przeprowadzonych eksperymentów wymienionych w rozdziale ‘Cel eksperymentów’. Wszystkie eksperymenty były przeprowadzone na bazie konfiguracji 3.2, czyli:

- ❖ MUT = 5%
- ❖ CROS = 60%
- ❖ POP = 300
- ❖ FES = 600 000

Wpływ prawdopodobieństwa mutacji na dokładność wyników programu

Opis:

Przyjęto następujący zestaw wartości parametru MUT do badania:

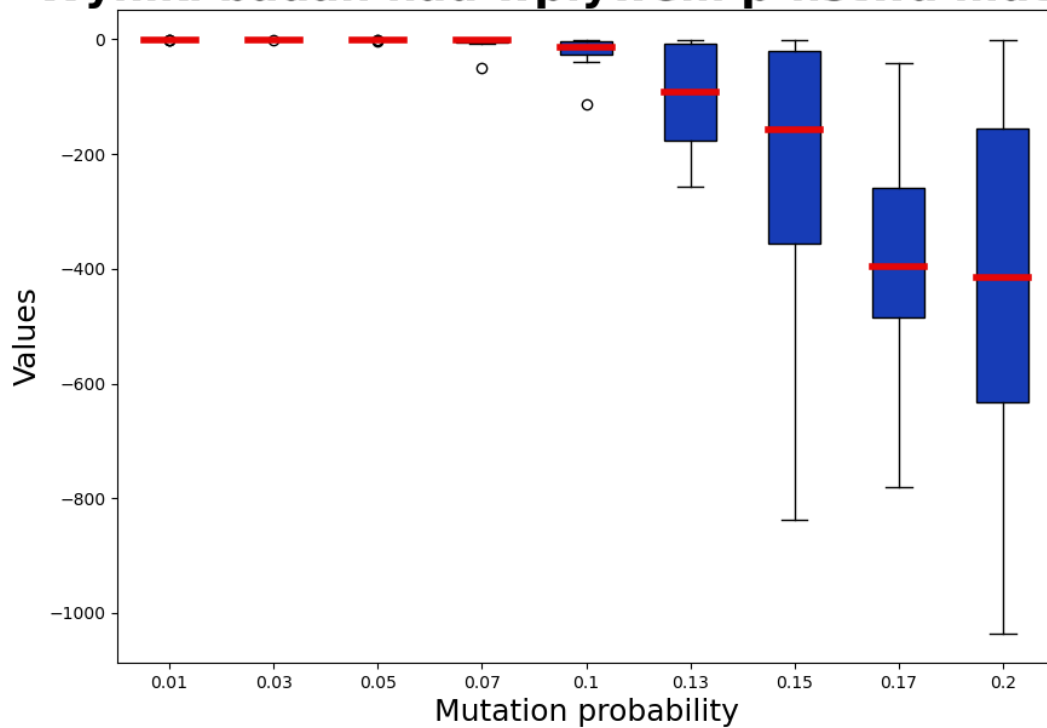
[1%, 3%, 5%, 7%, 10%, 13%, 15%, 17%, 20%]

Dla każdej z tych wartości MUT przeprowadzono dwadzieścia uruchomień programu. Wyniki tych uruchomień zapisano, a następnie utworzono z nich wykres porównujący medianę, odchylenie standardowe i rozrzut wyników programu dla danych wartości parametrów.

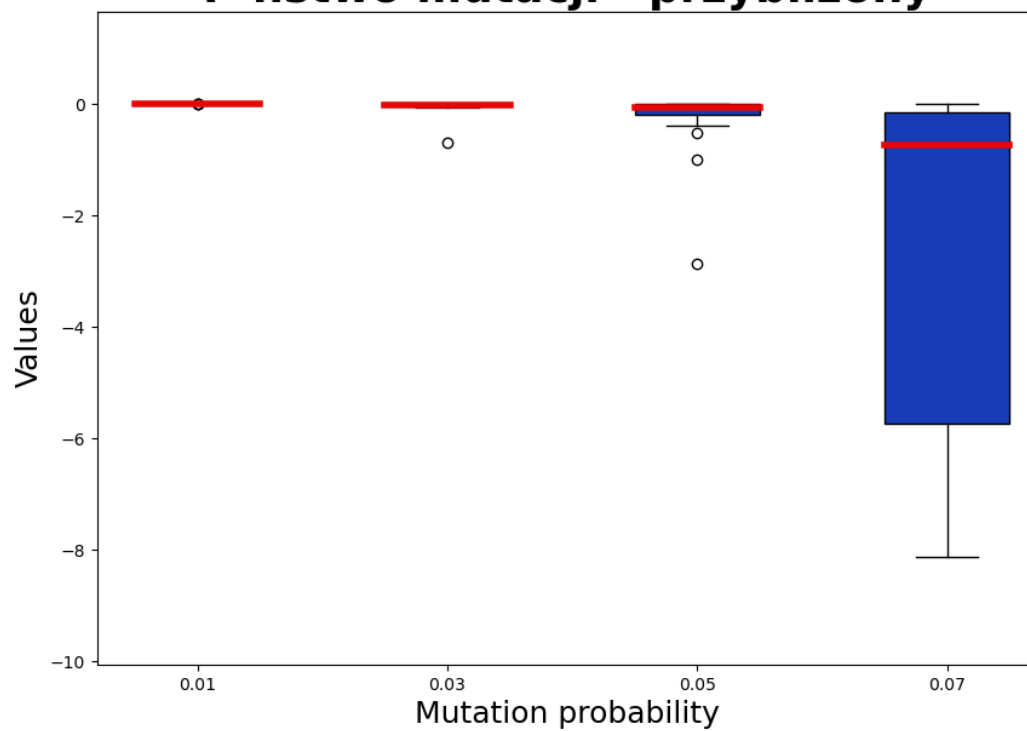
Oczekiwany wynik:

W związku z tym, iż program działa na algorytmie genetycznym większe wartości MUT mogą zniekształcić, a bardziej zaszkodzić działaniu tego programu. Jednakże zbyt niska wartość MUT, może doprowadzić do utknięcia na lokalnym optimum, gdyż nie będzie się dało przeskoczyć ‘górkę’. Dlatego przewiduję, że wyniki dla wyższych wartości MUT od 5% będą bardziej rozstrzelone, a dla niższych MUT poniżej 5% bardziej skupione, ale z możliwością utknięcia nie w tym optimum.

Wyniki badań nad wpływem p-ństwa mutacji



P-ństwo mutacji - przybliżony



Wnioski z wykresów:

Moje przewidywania okazały się słuszne. Począwszy od wartości $MUT = 7\%$ program wykazywał coraz to większe tendencje do zwracania gorszych wyników od poprzednich, nawet osiągając wartości około -1000 dla $MUT = 20\%$. Natomiast niespodziewanym było to, że dla niższych wartości MUT niż 5% dokładność zwracanych optimumów się zwiększyła.

Wnioski z eksperymentu:

Dla programu optymalizującego wykorzystującego algorytm genetyczny jako bazę wyższe wartości prawdopodobieństwa mutacji mocno szkodzą w poszukiwaniu optimum globalnego. Jak najmniejsze wartości MUT wpływają pozytywnie na uzyskiwane wyniki. Jednakże trzeba pamiętać, że mutacji nie można zlikwidować, gdyż jest ona kluczowa w wychodzeniu z 'dołków' czy 'górek', czyli wychodzeniu z lokalnego optimum.

Wpływ prawdopodobieństwa krzyżowania na dokładność wyników programu

Opis:

Przyjęto następujący zestaw wartości parametru CROS do badania:

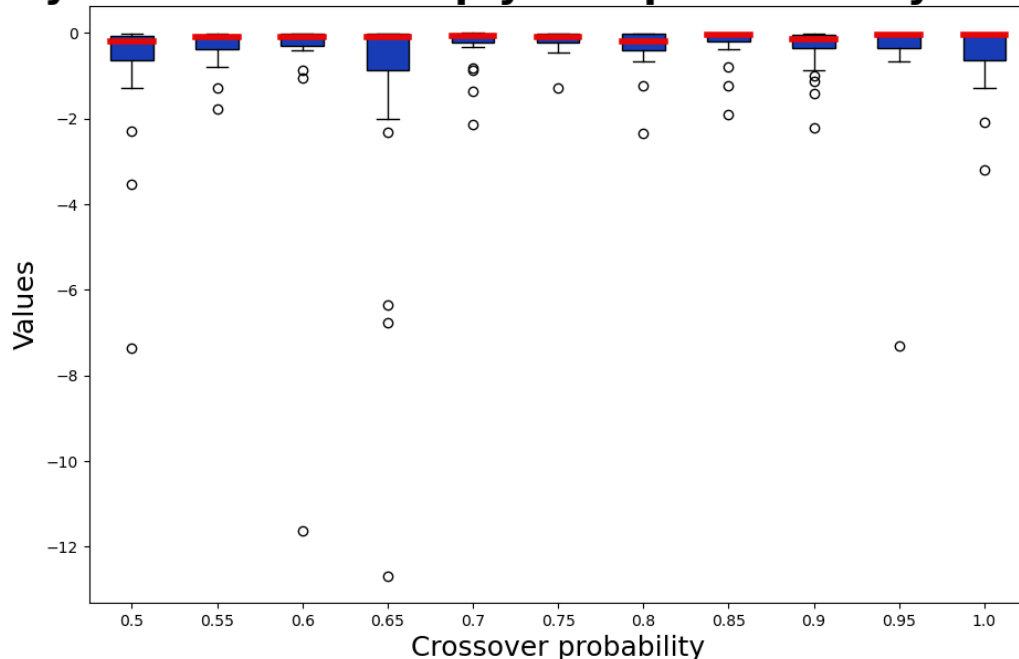
[50%, 55%, 60%, 65%, 70%, 75%, 80%, 85%, 90%, 95%, 100%]

Dla każdej z tych wartości CROS przeprowadzono dwadzieścia uruchomień programu. Wyniki tych uruchomień zapisano, a następnie utworzono z nich wykres porównujący medianę, odchylenie standardowe i rozrzut wyników programu dla danych wartości parametrów.

Oczekiwany wynik:

Prawdopodobieństwo krzyżowania odgrywa główną rolę w algorytmie krzyżowania, gdyż ono określa główny sposób tworzenia nowych generacji osobników. Zatem wartość tego parametru nie może być za niska, ale też nie może być za wysoka, aby nie za często gubić najbardziej obiecujących osobników.

Wyniki badań nad wpływem p-ństwa krzyżowania



Wnioski z wykresu:

Najlepsze wartości były osiągnięte dla prawdopodobieństw w przedziale 70% - 85%. Jednakże dla bazowego prawdopodobieństwa 60% wyniki też nie są złe. Ogólnie dla przedstawionych wartości wyniki są bardzo dobre oprócz wyników dla 50%, 65% i 100% CROS.

Wnioski z eksperymentu:

Jak to wynika z wykresu, możliwe wartości CROS są bardzo elastyczne dla przyjętej bazy. W obecnym przypadku jest na tyle dużo pokoleń, że niższe wartości CROS nie wpłyną znacząco na osiągnane wyniki. Jednakże przy mniejszej ilości następnych generacji niższe wartości CROS mogą być niewystarczające dla przyjętej dokładności. Koniec końców wartości CROS powinny się znajdować w przedziale od 55% do 85%.

Wpływ rozmiaru populacji na dokładność wyników programu

Opis:

Przyjęto następujący zestaw wartości parametru POP do badania:

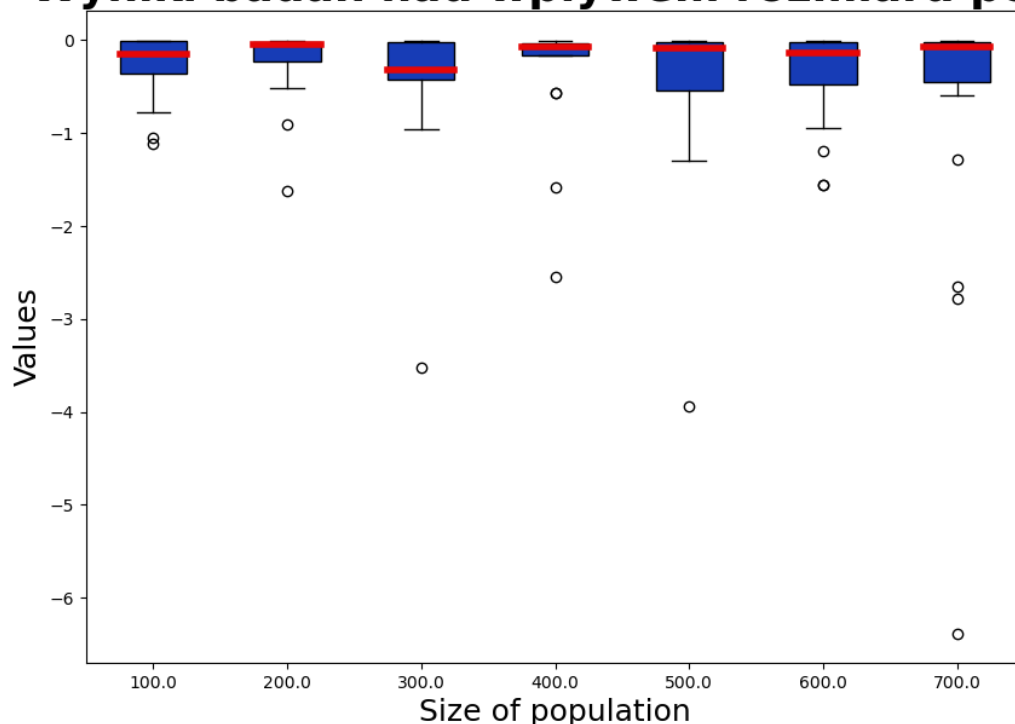
[100, 200, 300, 400, 500, 600, 700]

Dla każdej z tych wartości FES przeprowadzono dwadzieścia uruchomień programu. Wyniki tych uruchomień zapisano, a następnie utworzono z nich wykres porównujący medianę, odchylenie standardowe i rozrzut wyników programu dla danych wartości parametrów.

Oczekiwany wynik:

Mała liczba osobników w populacji na pewno zwiększy ilość nowych pokoleń, co może przyczynić się do pewnego znalezienia optimum. Jednakże te założenie jest zasadne, jeśli trafimy na odpowiednich osobników, którzy nakierują nas na odpowiednich osobników. Natomiast duża populacja będzie oznaczać małą liczbę pokoleń osobników co zmniejszy szansę na znalezienie odpowiedniego rozwiązania, pomimo tego że większą populacją pokrywamy większy obszar przeszukiwań.

Wyniki badań nad wpływem rozmiaru pop.



Wnioski z wykresu:

Z wykresu jasno wynika, że najlepszy wynik został osiągnięty dla POP o wartości 400, a drugi najlepszy dla POP równym 200. Reszta wyników też nie jest taka zła, lecz można zauważyć większą niestabilność wyników dla POP o wartości 700.

Wnioski z eksperymentu:

Z tego eksperymentu można wyciągnąć bardzo ciekawy wniosek na temat wielkości populacji. Zaobserwowano ewidentną korelację między wielkością populacji a ilością bitów (400 bitów) w pojedynczym osobniku (czyli 400 wymiarów). Można z tego wysnuć wniosek, że wartość POP powinna być wybierana zgodnie z liczbą wymiarów

pojedynczego osobnika, gdzie te wartości to wielokrotności liczby wymiarów (w naszym przypadku: 200, 400, 800, itd.)

Wpływy ilości danego FES-u na dokładność wyników programu

Opis:

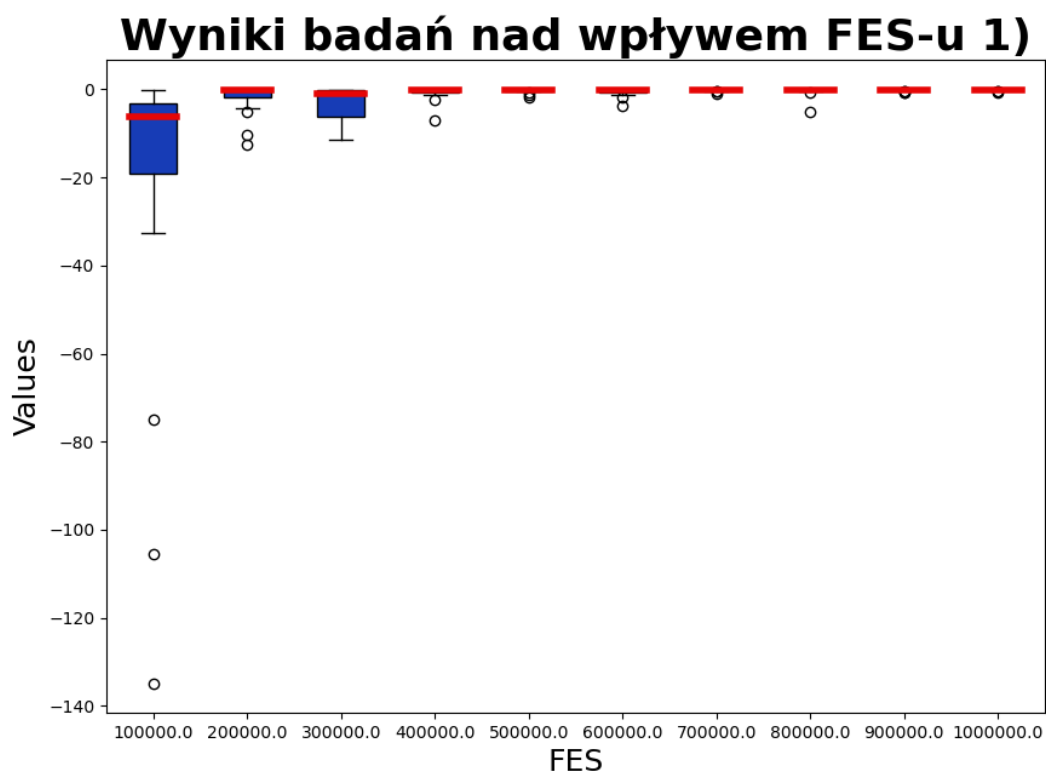
Przyjęto następujący zestaw wartości parametru FES do badania:

[100 000, 200 000, 300 000, 400 000, 500 000, 600 000, 700 000, 800 000, 900 000, 1 000 000]

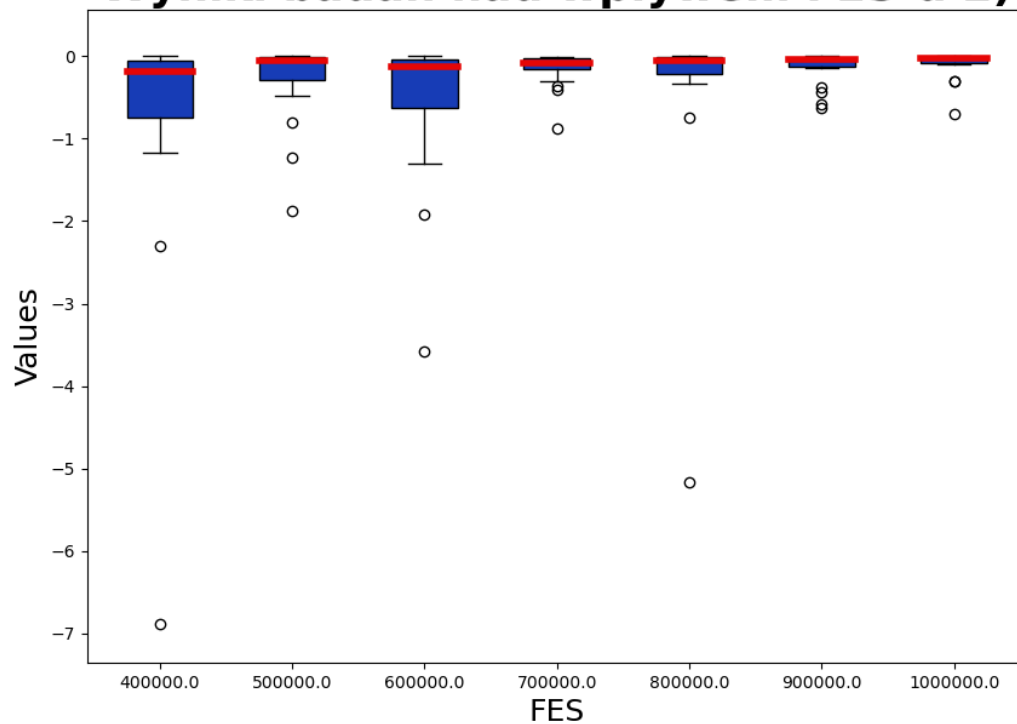
Dla każdej z tych wartości FES przeprowadzono dwadzieścia uruchomień programu. Wyniki tych uruchomień zapisano, a następnie utworzono z nich wykres porównujący medianę, odchylenie standardowe i rozrzut wyników programu dla danych wartości parametrów.

Oczekiwany wynik:

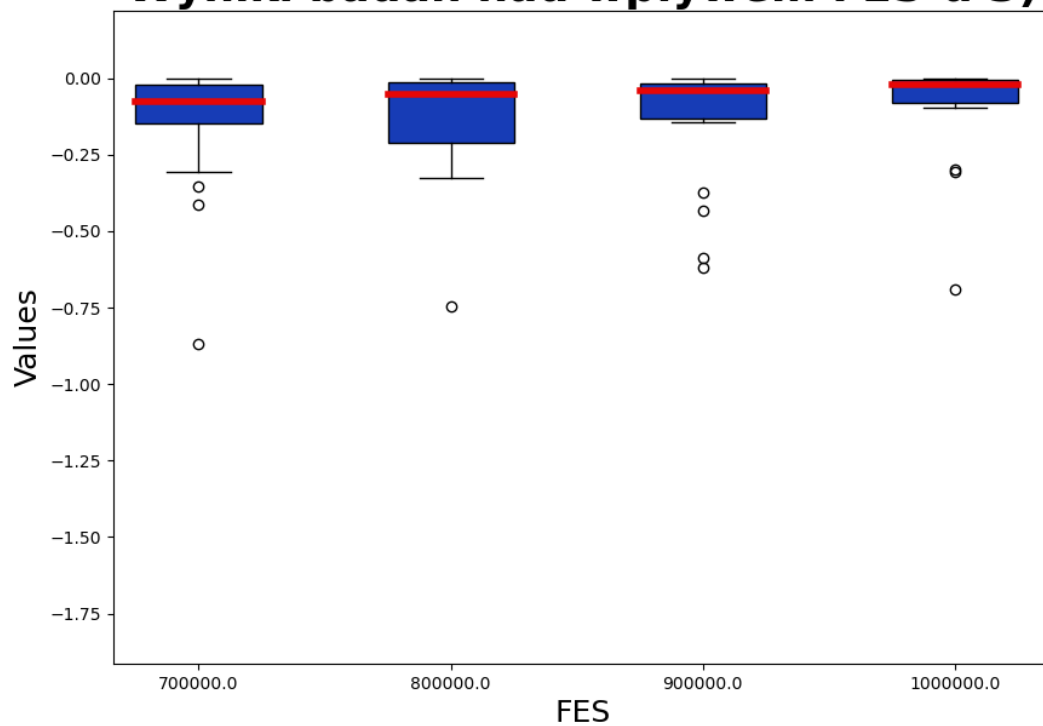
Wraz ze zwiększeniem naszego budżetu FES wyniki powinny być coraz bardziej dokładne, w związku ze zwiększeniem ilości generacji w których zajdzie ewolucja.



Wyniki badań nad wpływem FES-u 2)



Wyniki badań nad wpływem FES-u 3)



Wnioski z wykresów:

Pierwszy wykres przedstawia wyniki dla poszczególnych wartości FES-u, gdzie następne dwa skupiają się na coraz mniejszych wycinkach badanych parametrów. Widać tu ewidentną tendencję do bardziej skupionych wyników przy wyższych wartościach FES-u.

Wnioski z eksperymentu:

Tak jak przewidziałem, wraz ze zwiększeniem budżetu FES uzyskiwane wartości stają się coraz to bardziej dokładne, gdzie sporą różnicę można zauważyć już od 400 000 FES. Jednakże nie można w ślepo przydzielić zbyt dużo FES-u, gdyż może wyższe wartości oznaczają lepszą dokładność to jednak zużywają duże ilości czasu i zasobów.

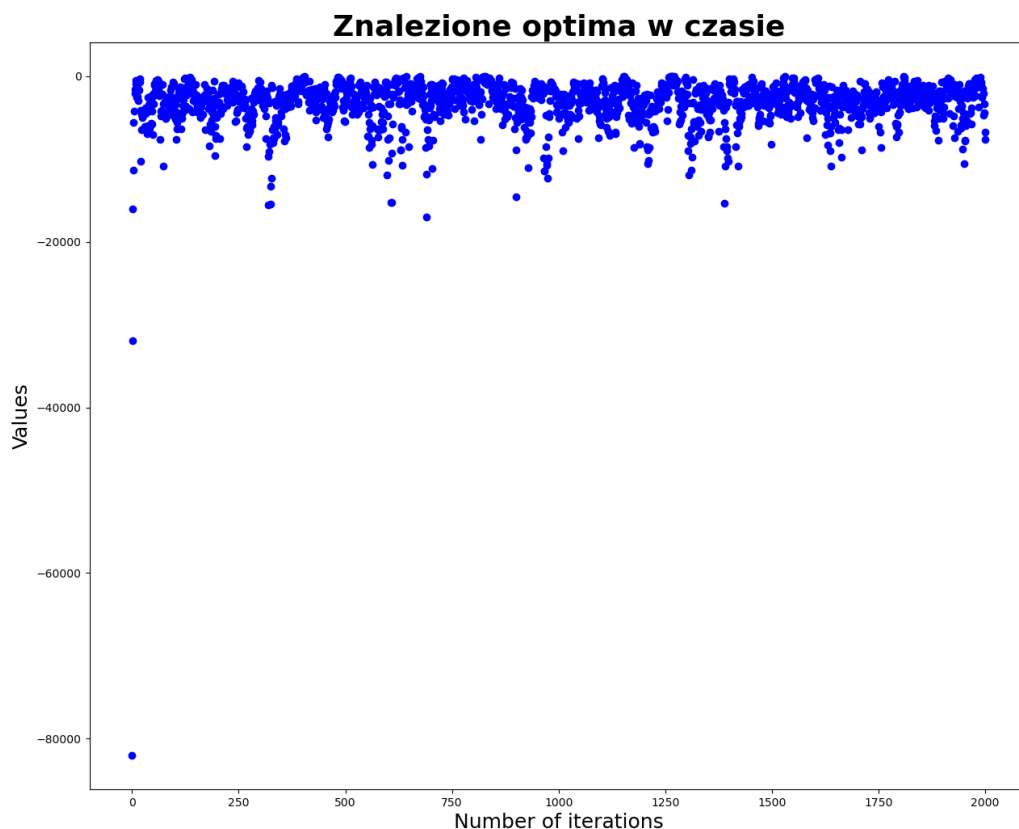
Badanie przebiegu znajdowania optimum w czasie

Opis:

Eksperyment ten ma na celu przeanalizowanie jaką drogę przebywa nasza populacja w czasie działania programu w celu znalezienia optimum. Ważne jest to, że nie mapuję każdego osobnika, lecz same znalezione optima w danym punkcie przebiegu programu, gdyż wykres przedstawiający wszystkich osobników jest nieczytelny i daje słabe wejście w działanie programu. Jednakże przedstawię taki wykres w celu wyjaśnienia dlaczego tak jest

Oczekiwany wynik:

Wygenerowany wynik na podstawie bazowej konfiguracji przedstawi drogę znajdowania optimum globalnego. Na samym początku powinny się pojawić dość niskie wartości, aby dość szybko odkryć lepsze optima i zacząć krążyć wokół nich. Potem do końca programu oscylowanie będzie odbywało się w otoczeniu optimum globalnego, aby móc je znaleźć.



Wnioski:

Zgodnie z wcześniejszymi założeniami stało się to co przewidziałem. Jedyny fakt, który dziwi to szybkość z jaką udało się dostać do wyższych wartości optimum na samym początku działania programu. Jednakże zwracając fakt na to, że zaczynaliśmy z dużą populacją o prawdziwie losowych wartościach, fakt nagłego wzrostu na wykresie nie jest już aż tak szokujący.

Ogólnie można stwierdzić, że przez większość czasu program skupiał się na znalezieniu jak najlepszego optimum.

Adnotacja do wykresów wszystkich osobników:

Jak widać na załączonym wycinku wykresu ciężko przedstawić wszystkich osobników przy tak dużej populacji. Wiele z osobników na siebie nachodzi. Oczywiście jest możliwe odnotowanie większej ilości osobników występujących w górnych warstwach wykresu, co udowadnia wcześniej ustalone fakty, lecz ten wykres jest mniej czytelny.

Taki sposób przedstawienia danych może lepiej by się sprawdzał dla mniejszych populacji, lecz dla mojej wybranej bazy jest nieodpowiedni.

